C4 Ingestion and orchestration

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2
C4 Ingestion and orchestration 3
C4-L1 Choosing how data gets into Fabric 3
C4-L2 Incremental copy with a watermark, or with CDC 7
C4-L3 Pipelines that fail fast, retry sensibly and alert 11
Track C best-practice checklist 16
Appendix: solutions and answer keys 17

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C4 Ingestion and orchestration

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Data Factory pipelines, Dataflows Gen2 and copy jobs.

C4-L1 Choosing how data gets into Fabric

Concept

Fabric offers several ways to move data. They overlap, and picking by habit creates pipelines nobody needs. Microsoft Learn's data movement guide (updated April 2026) gives a starting rule by layer:

- **Curated data from an existing system, for reporting: Mirroring.** If the source already does its own ETL and you need the result in Fabric, mirroring is the simplest and cheapest choice (lesson C3-L2).
- **Raw ingestion into bronze: Copy job.** It handles bulk copy, incremental copy and change data capture (CDC), with scheduling and state tracking built in, without building a pipeline.
- **Low-latency events: Eventstream** (lesson C7-L1).

Then add tools only where they earn their place:

- **Pipeline** (Data Factory): orchestration. Run activities in order, branch on success or failure, loop, retry, and call notebooks, stored procedures, dataflows and Copy jobs. A pipeline with a copy activity also moves data, but you manage incremental state yourself.
- **Dataflow Gen2**: Power Query, low-code transformation for analysts, with a data destination per query.
- **Spark notebook**: code-first transformation at scale, the closest match to your Databricks work (lesson C5-L1).

A simple way to decide:

1. Does the source need to be replicated as a whole database? Mirroring.
2. Is it a copy of tables or files into bronze? Copy job.
3. Are there events that must arrive within seconds? Eventstream.
4. Is there transformation? Spark for engineers, Dataflow Gen2 for low-code authors.
5. Do several steps need ordering, branching and alerts? Wrap them in a pipeline.

Two warnings. A pipeline that only copies tables re-creates what a Copy job does, with more to maintain. And Dataflow Gen2 is not a replacement for Spark on large volumes. Check your capacity usage (lesson C10-L3) before you standardise on one tool.

Why it matters

A team built 40 pipelines, each with a copy activity, a lookup of the last loaded timestamp from a control table, and a stored procedure to update it. Two of them had an off-by-one error in the watermark and silently skipped rows every night. A Copy job per source would have tracked incremental state itself, with nothing for the team to get wrong.

Worked example: weak code

```
❶ Source: Azure SQL CRM (200 tables)   -> 200 pipelines, copy activity +
     control table
❷ Source: Clickstream events            -> hourly copy activity from storage
❸ Transform: Dataflow Gen2 on 400 M rows per day
```

1. **Line 1.** Hand-built incremental state for a whole database: mirroring or a Copy job does this for you.
2. **Line 2.** Events that need low latency, handled as hourly batches.
3. **Line 3.** A low-code tool on engineering-scale volume. Spark suits this better.

Worked example: strong code

```
❶ Source: Azure SQL CRM (200 tables)   -> Mirroring (curated reporting copy)
❷ Source: Supplier files in ADLS        -> Copy job, incremental, into
     bronze
  3 Source: Clickstream events           -> Eventstream into an eventhouse
  4 Transform: Spark notebooks, bronze to gold
❸ Orchestration: one pipeline runs the notebooks in order, retries, alerts
     on failure
```

1. **Line 1.** Free replication compute, no incremental logic to maintain.
2. **Line 2.** State tracking built in.
3. **Line 5.** The pipeline orchestrates. It does not re-create copying.

Mirroring for curated databases, Copy job for bronze, Eventstream for events, Spark or Dataflow Gen2 for transformation, and pipelines to orchestrate.

Recap card

- **Microsoft's starting rule by layer** Gold from existing systems: Mirroring. Bronze ingestion: Copy job. Low-latency events: Eventstream.

- **What a pipeline is for** Orchestration: ordering, branching, loops, retries and alerts across activities.
- **Spark or Dataflow Gen2?** Spark for code-first, large-volume engineering. Dataflow Gen2 for low-code Power Query authors.
- **Copy activity vs Copy job for incremental loads** Copy activity needs hand-built state. Copy job tracks the last successful run itself.

Practice

Exercise 1. Multiple choice (level 1)

You need raw CSV deliveries from ADLS copied into a bronze lakehouse every hour, only new files each time. Which tool does Microsoft Learn recommend starting with?

- a) Mirroring
- b) Copy job
- c) Dataflow Gen2
- d) A pipeline with a copy activity and a control table

Why?

- r1) Copy job supports incremental copy and scheduling natively for bronze ingestion.
- r2) Copy job is the only tool that reads ADLS.

Exercise 2. Architecture design (level 2)

Pick the tool for each need.

Options: Mirroring, Copy job, Eventstream, Spark notebook, Pipeline.

1. An ERP database's curated tables, kept current for reporting
2. Sensor events that must trigger alerts within seconds
3. Joining and cleaning 300 M rows a day into silver
4. Run copy, then three notebooks in order, and email on failure

Exercise 3. Put in order (level 2)

Order the questions in the decision sequence from this lesson.

- Transformation needed, and by whom?
- Several steps that need ordering and alerts?
- Copy tables or files into bronze?
- Replicate a whole database or catalog?
- Events that must arrive within seconds?

Exercise 4. Write the function (level 3)

Write `recommend(need: str) -> str` encoding the decision rule. Accepted needs and answers:

- `"replicate_database"` → `"Mirroring"`
- `"bronze_copy"` → `"Copy job"`
- `"low_latency_events"` → `"Eventstream"`
- `"code_transform"` → `"Spark notebook"`
- `"lowcode_transform"` → `"Dataflow Gen2"`
- `"orchestrate"` → `"Pipeline"`

Raise `ValueError` for anything else, with the unknown need in the message.

Your code must pass these checks:

- Each need maps to one tool, as listed.
- Unknown needs must raise ValueError naming the need.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
TOOL_FOR_NEED = ...

# Fabric tool for a data movement or processing need.
def recommend(need: str) -> str: ...
```

Starter code:

```
def recommend(need):
    return "Pipeline"
```

Exercise 5. Multiple choice (level 4)

A team copies 60 tables from Azure SQL into bronze with 60 pipelines, each using a copy activity, a control table for the last watermark, and a stored procedure to update it. Two tables lose rows every night. What change removes the whole class of bug?

- a) Add a retry to each copy activity.
- b) Move the tables to Copy jobs with incremental copy, which track state themselves.
- c) Run the pipelines twice a night.
- d) Replace the stored procedure with a Dataflow Gen2.

Why?

- r1) Copy job manages the state of the last successful run, so there is no hand-written watermark logic to get wrong.

- r2) Pipelines cannot copy incrementally.

Sources

- Data movement guide: Mirroring for gold from existing systems, Copy job for bronze ingestion, Eventstream for low-latency events; copy activity requires managing incremental state yourself. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/decision-guide-data-movement (updated 2026-04-13))
- Copy job supports full copy, watermark-based and CDC-based incremental copy, and tracks the state of the last successful run. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job (updated 2026-09-17))

C4-L2 Incremental copy with a watermark, or with CDC

Concept

A **Copy job** copies data in one of two modes. **Full copy** moves everything on every run. **Incremental copy** moves everything on the first run, then only new or changed data. The job tracks the state of the last successful run itself (Microsoft Learn, checked September 2026).

Incremental copy works in one of two ways:

- **Watermark.** You pick an incremental column, such as a `ROWVERSION`, a datetime, a date or an increasing integer. Each subsequent load copies rows whose value is **larger than any value copied before**.
- **CDC** (preview in Copy job). If change data capture is enabled on the source database, the job reads its change stream and copies every insert, update and delete since the last run. You do not choose a column.

The watermark method has two blind spots, and both follow from "larger than":

- It **cannot see deletes**. A deleted row simply stops appearing, and the destination keeps it.
- It **misses late rows**. A row committed after the run, with a watermark value at or below the one already copied, is never picked up. A `modified_at` set by the application clock, or several rows written in the same second, causes this.

Prefer CDC when you need deletes, or a `ROWVERSION` column, which the database increments on every change. Microsoft also lets you reset incremental copy to a full copy, for the whole job or per table, to recover.

Choose the **update method** for the destination on purpose. The default is **Append**, which keeps full history, and appends every changed row again as a new row. **Merge** updates existing rows by key (the primary key by default), **Overwrite** replaces the data, and **SCD Type 2** keeps versioned history. For a silver table that must hold one row per key, choose Merge.

Why it matters

An order table was copied incrementally on `modified_at`, set by the ordering app. At 23:59:59 one batch committed after the nightly run had read up to 23:59:59. Those rows carried the same timestamp as the last copied row, so they were never picked up. Finance found 212 missing orders at month end. CDC, or a `ROWVERSION` column, would have caught them.

Worked example: weak code

```
1  def next_batch(rows, last_wm):
1      batch = [r for r in rows if r["modified_at"] >= last_wm]
2      return batch, max(r["modified_at"] for r in batch)
```

1. **Line 2.**

   = re-copies the last run's boundary rows every time, which Append turns into duplicates.

2. **Line 3.** Fails with ValueError on a run with no new rows.

Worked example: strong code

```
1  Row = dict[str, int]
2
3
4  def next_batch(rows: list[Row], last_wm: int | None) -> tuple[list[Row],
       int | None]:
5      """Rows with a watermark above last_wm, and the new watermark."""
1      batch = [r for r in rows if last_wm is None or r["rv"] > last_wm]
7      if not batch:
2          return [], last_wm
9      return batch, max(r["rv"] for r in batch)
```

1. **Line 6.** Strictly greater, as Copy job does, on a ROWVERSION-style column.
2. **Line 8.** An empty run keeps the old watermark instead of failing.

Watermarks copy rows above the last value seen: no deletes, no late rows. Use CDC or a ROWVERSION column, and Merge for one row per key.

Recap card

- **Copy job incremental copy** First run copies everything. Later runs copy new or changed data. State is tracked by the job.
- **Watermark blind spots** Cannot see deletes. Misses late rows at or below the stored watermark.
- **When to use CDC** When deletes must replicate, or changes arrive out of order.
- **Copy job update methods** Append (default), Merge, Overwrite, SCD Type 2. Merge for one row per key.

Practice

Exercise 1. Multiple choice (level 1)

A Copy job uses watermark-based incremental copy on `modified_at`. A row is deleted in the source. What happens in the destination?

- a) The row is deleted on the next run.
- b) Nothing. Watermark-based incremental copy cannot detect deletes.
- c) The row is marked with a soft-delete flag.
- d) The job fails until you reset it.

Why?

- r1) A watermark only finds rows with a larger value. A deleted row has no value to find.
- r2) Deletes are never supported by Copy job.

Exercise 2. Predict the output (level 2)

Run 1 copied up to watermark 105. A late row with `rv` 104 commits afterwards. What does run 2 copy?

```
1 rows = [{"id": 1, "rv": 101}, {"id": 2, "rv": 105}, {"id": 3, "rv": 104},
    {"id": 4, "rv": 107}]
2 last_wm = 105
3 batch = [r["id"] for r in rows if r["rv"] > last_wm]
4 print(batch)
```

Exercise 3. Fill the gap (level 2)

Complete the Copy job settings for a silver table that must hold one current row per `order_id` and reflect source deletes.

```
1 Incremental method: _____(1)
2 Update method: _____(2)
```

Exercise 4. Write the function (level 3)

Write `next_batch(rows, last_wm)` for a hand-built incremental load that behaves like Copy job's watermark method. `rows` is a list of dicts with an integer `rv`. `last_wm` is `None` before the first run. Return `(batch, new_wm)`: rows with `rv` strictly greater than `last_wm` (all rows when `last_wm` is `None`), and the largest `rv` in the batch. On an empty batch return `([], last_wm)`.

Your code must pass these checks:

- With last_wm None, copy every row and return the largest rv.
- Copy only rows strictly greater than the watermark. >= re-copies the boundary row.
- An empty batch keeps the previous watermark and must not raise.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Rows above the watermark, and the new watermark.
def next_batch(
    rows: list[Row],
    last_wm: int | None,
) -> tuple[list[Row], int | None]: ...
```

Starter code:

```python
def next_batch(rows, last_wm):
    batch = [r for r in rows if r["rv"] >= last_wm]
    return batch, max(r["rv"] for r in batch)
```

Exercise 5. Architecture design (level 4)

Configure incremental copy of an orders table into silver. Orders are updated and sometimes deleted. The source has CDC enabled.

Options: CDC, Watermark on modified_at.

1. Incremental method
2. Update method
3. After an outage caused missed changes

Sources

- Copy job modes, watermark column types, strictly larger watermark rule, CDC captures deletes, watermark cannot detect deletes, reset to full copy per job or table. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/incremental-copy-job (updated 2026-09-15) and .../what-is-copy-job (updated 2026-09-17))

- Update methods Append (default), Merge (primary key by default), Overwrite and SCD Type 2. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job (updated 2026-09-17))
- CDC replication in Copy job is in preview. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job (updated 2026-09-17))

C4-L3 Pipelines that fail fast, retry sensibly and alert

Concept

A Data Factory **pipeline** runs activities: Copy job, notebook, stored procedure, Dataflow, If Condition, ForEach, Until, and more. By default a pipeline holds up to 120 activities. Three settings on each activity decide how a failure behaves (Microsoft Learn, checked September 2026):

- **Timeout.** How long an activity may run before it is stopped. The default is **12 hours**, and the maximum is seven days, written `D.HH:MM:SS`. A notebook that normally takes 10 minutes and hangs will hold its slot, and its capacity, for 12 hours unless you lower this.
- **Retry.** Retries happen only if you enable them. The retry count then defaults to 1, with **30 seconds** between attempts. Retry steps that fail for passing reasons, such as a busy source or a network drop. Do not retry a step that failed because its data is wrong. It will fail again, later.
- **Dependency branches.** Each activity connects to the next through a condition: **Upon Success**, **Upon Failure**, **Upon Completion** (either way) or **Upon Skip**. Put alerts and clean-up on Upon Failure. Upon Completion runs even when the step succeeded.

Parameters and expressions keep one pipeline reusable across sources and environments. Expressions start with `@`: `@pipeline().parameters.wave`. Inside a string, `@{...}` interpolates: `"landing/@{pipeline().parameters.wave}/"`. The result of interpolation is always a **string**, so `"@{pipeline().parameters.max_rows}"` passes text, not a number. When a setting needs a number, use the bare expression `@pipeline().parameters.max_rows`.

Keep business logic out of the pipeline. Transformations belong in notebooks and SQL, where they can be tested (lesson A9-L1). The pipeline decides order, retries and alerts.

Why it matters

A nightly pipeline ran a 10-minute notebook that sometimes hung on a locked source. With the default 12-hour timeout and no failure branch, each hang held its capacity until mid-morning. Downstream reports showed yesterday's data, and nobody was told. A 30-minute timeout, one retry and an Upon Failure email would have raised the alarm at 02:30 instead of 10:00.

Worked example: weak code

```
1 Activity: nb_silver_orders (notebook)
1   Timeout: default
3   Retries: not enabled
2 Activity: nb_gold_sales  depends on nb_silver_orders: Upon Completion
3 Activity: send_failure_email  depends on nb_gold_sales: Upon Completion
```

1. **Line 2.** 12 hours for a 10-minute job: a hang goes unnoticed until morning.
2. **Line 4.** Gold runs even when silver failed, publishing stale or partial data.
3. **Line 5.** The 'failure' email is sent every night, so people learn to ignore it.

Worked example: strong code

```
1 Activity: nb_silver_orders (notebook)
1   Timeout: 0.00:30:00
3   Retries: enabled, 1 retry, 60 seconds apart
2 Activity: nb_gold_sales  depends on nb_silver_orders: Upon Success
3 Activity: send_failure_email  depends on nb_silver_orders: Upon Failure
6 Activity: send_failure_email_gold  depends on nb_gold_sales: Upon Failure
```

1. **Line 2.** Three times the normal run time: a hang fails within half an hour.
2. **Line 4.** Gold runs only on good silver.
3. **Line 5.** Alerts fire only when something failed.

Set timeouts near the real run time, retry only transient failures, chain on Upon Success and alert on Upon Failure.

Recap card

- **Activity timeout default** 12 hours, maximum seven days, format D.HH:MM:SS. Set it near the real run time.
- **Activity retries** Only when enabled. Count defaults to 1, interval 30 seconds. Retry transient failures only.
- **Dependency branches** Upon Success, Upon Failure, Upon Completion (either way), Upon Skip.
- **@{...} vs @...** @{...} inside text always yields a string. A bare @ expression keeps the value's type.

Practice

Exercise 1. Multiple choice (level 1)

You leave an activity's timeout at its default. How long can it run before Fabric stops it?

- a) 30 minutes
- b) 1 hour
- c) 12 hours
- d) 7 days

Why?

- r1) The documented default timeout is 12 hours. Seven days is the maximum you can set.
- r2) Timeouts match the capacity's pause schedule.

Exercise 2. Multiple choice (level 2)

An alert activity is attached to a notebook with **Upon Completion**. When does it run?

- a) Only when the notebook fails
- b) Only when the notebook succeeds
- c) Whether the notebook succeeds or fails
- d) Only when the notebook is skipped

Why?

- r1) Upon Completion runs after success or failure. Upon Failure is the failure-only branch.
- r2) Completion means the capacity finished all jobs.

Exercise 3. Fill the gap (level 2)

Build a folder path from the pipeline parameter `wave`, and pass the numeric parameter `max_rows` as a number.

```
Folder path: landing/_____(1)pipeline().parameters.wave}/
Max rows:    _____(2)pipeline().parameters.max_rows
```

Exercise 4. Write the function (level 3)

Write `parse_timeout(value: str) -> int` for a pipeline linter. It parses the `D.HH:MM:SS` format and returns the total seconds. Raise `ValueError` if the format is wrong, if hours are above 23 or minutes or seconds above 59, or if the total exceeds 7 days.

What you need to know.

Python's `re` module matches text against a pattern:

- `re.compile(r"...")` builds a pattern once, so you reuse it.
- `\d` matches one digit, `\d{2}` exactly two, `\d+` one or more, and `\.` a literal dot.
- Brackets `( )` **capture** part of the match.
- `pattern.fullmatch(text)` matches the **whole** string and returns a match object, or `None` when the text does not fit.
- `match.groups()` returns the captured parts as strings.

```python
import re

MONTH = re.compile(r"(\d{4})-(\d{2})")
MONTH.fullmatch("2026-03").groups()   # ('2026', '03')
MONTH.fullmatch("2026-03x")           # None
```

Your code must pass these checks:

- Convert days, hours, minutes and seconds to seconds.
- Reject a missing day part, missing fields, non-numbers and out-of-range hours or minutes.
- The maximum timeout is seven days.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
TIMEOUT_PATTERN = ...

# Module constant.
MAX_SECONDS = ...

# Seconds in a pipeline activity timeout written as D.HH:MM:SS.
def parse_timeout(value: str) -> int: ...
```

Starter code:

```python
def parse_timeout(value):
    days, rest = value.split(".")
    h, m, s = rest.split(":")
    return int(days) * 86400 + int(h) * 3600 + int(m) * 60 + int(s)
```

Exercise 5. Spot the bug (level 4)

This pipeline publishes gold tables even when silver fails. Tap the line that causes it, then rewrite the dependencies so gold runs only on success and an alert fires on failure.

```
Activity: nb_silver_orders
```

```
  Timeout: 0.00:30:00
Activity: nb_gold_sales  depends on nb_silver_orders: Upon Completion
Activity: email_on_failure  depends on nb_silver_orders: Upon Failure
```

Write down the line number of the bug. Then write the fix.

Sources

- Activity timeout defaults to 12 hours with a seven-day maximum in D.HH:MM:SS; retries when enabled default to 1 with a 30-second interval; up to 120 activities per pipeline; UponSuccess, UponFailure, UponCompletion and UponSkip branches. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/activity-overview (updated 2026-08-10))
- Pipeline expressions use @pipeline().parameters; string interpolation with @{...} always returns a string. (verified: https://learn.microsoft.com/en-us/fabric/data-factory/expression-language (updated 2024-12-18))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C4-L1 Choosing how data gets into Fabric

C4-L1 exercise 1

Hints. (1) Which layer is this? (2) Which tool tracks incremental state itself? (3) Copy job.

Answer. b) Copy job Because: r1) Copy job supports incremental copy and scheduling natively for bronze ingestion.

Bronze ingestion starts with Copy job: incremental copy without building a pipeline.

Common wrong answers.

- `a`: Mirroring replicates databases and catalogs. This is file ingestion into bronze.
- `c`: Dataflow Gen2 is for low-code transformation. Nothing needs transforming yet.
- `d`: It works, but you build and maintain the incremental state that Copy job tracks for you.
- `b|r2`: Right answer, wrong reason. Pipelines and shortcuts read ADLS too.

C4-L1 exercise 2

Hints. (1) Replicate, stream, transform or orchestrate? (2) Which tool is event-driven? (3) Which tool coordinates other tools?

Answer.

- Curated database tables: mirroring.
- Low-latency events: Eventstream.
- Heavy transformation: Spark.
- Ordering, branching and alerts across steps: a pipeline.

Each tool has a lane. Stay in it and there is less to build and maintain.

Common wrong answers.

- erp = `copyjob`: A Copy job works, but mirroring replicates the whole database with free replication compute and no schedule to manage.

- iot = `copyjob`: Copy jobs run on schedules. Seconds-level events need Eventstream.
- orchestrate = `spark`: A notebook can call others, but retries, branches and alerts across steps are what pipelines are for.

C4-L1 exercise 3

Hints. (1) Movement first. (2) Transformation next. (3) Orchestration wraps the rest.

Answer.

1. Replicate a whole database or catalog?
2. Copy tables or files into bronze?
3. Events that must arrive within seconds?
4. Transformation needed, and by whom?
5. Several steps that need ordering and alerts?

Choose movement tools, then transformation, then wrap them in orchestration only where needed.

Common wrong answers.

- Starting with a pipeline leads to building copy logic inside it. Choose the movement tools first, then orchestrate them.
- Decide how data arrives before how it is transformed.

C4-L1 exercise 4

Hints. (1) A dictionary from need to tool. (2) Check membership before returning. (3) Put the need in the error message.

Answer.

```python
TOOL_FOR_NEED: dict[str, str] = {
    "replicate_database": "Mirroring",
    "bronze_copy": "Copy job",
    "low_latency_events": "Eventstream",
    "code_transform": "Spark notebook",
    "lowcode_transform": "Dataflow Gen2",
    "orchestrate": "Pipeline",
}


def recommend(need: str) -> str:
    """Fabric tool for a data movement or processing need.

    Raises:
        ValueError: If the need is not in the table.
    """
```

```
    if need not in TOOL_FOR_NEED:
        raise ValueError(f"unknown need: {need}")
    return TOOL_FOR_NEED[need]
```

An explicit table makes the team's convention reviewable, and a clear error beats a silent default.

C4-L1 exercise 5

Hints. (1) Where does the bug live? (2) In hand-written state tracking. (3) Which tool does that for you?

Answer. b) Move the tables to Copy jobs with incremental copy, which track state themselves. Because: r1) Copy job manages the state of the last successful run, so there is no hand-written watermark logic to get wrong.

Removing hand-built incremental state removes the bugs it causes. Copy job tracks the last successful run itself.

Common wrong answers.

- `a`: Retries repeat the same logic, including its watermark bug.
- `c`: More runs repeat the off-by-one error more often.
- `d`: That moves the hand-built state into another tool without removing it.
- `b|r2`: Right answer, wrong reason. Pipelines can copy incrementally, but you build the state tracking.

C4-L2 Incremental copy with a watermark, or with CDC

C4-L2 exercise 1

Hints. (1) What does a watermark look for? (2) Does a deleted row have a new value? (3) CDC captures deletes.

Answer. b) Nothing. Watermark-based incremental copy cannot detect deletes. Because: r1) A watermark only finds rows with a larger value. A deleted row has no value to find.

Watermarks only see rows that exist with a larger value. Use CDC when deletes matter.

Common wrong answers.

- `a`: That needs CDC, which captures delete events.
- `c`: Soft deletes come from CDC with SCD Type 2. A watermark sees nothing.
- `d`: The job succeeds. The deleted row just stays in the destination.

- `b|r2`: Right answer, wrong reason. CDC-based incremental copy does replicate deletes.

C4-L2 exercise 2

Hints. (1) Which rows have rv above 105? (2) Where does row 3 fall? (3) Below the watermark.

Answer.

```
[4]
```

Only row 4 is copied. Row 3 is lost for good unless you reset to a full copy, which is why commit-order columns such as ROWVERSION, or CDC, are safer.

Common wrong answers.

- `[3, 4]`: Row 3 has rv 104, below the watermark 105, so it is never copied. That is the late-row blind spot.

C4-L2 exercise 3

Hints. (1) Which method captures deletes? (2) Which update method keeps one row per key? (3) `CDC` and `Merge`.

Answer.

```
Incremental method: CDC
Update method: Merge
```

CDC replicates inserts, updates and deletes. Merge keeps one row per key.

Common wrong answers.

- g2 = `Append`: Append adds each changed row again, so a key appears many times.
- g1 = `Watermark`: A watermark cannot see deletes.

C4-L2 exercise 4

Hints. (1) Handle last_wm None first. (2) Strictly greater than, to avoid re-copying the boundary. (3) Return ([], last_wm) when nothing is new.

Answer.

```python
Row = dict[str, int]


def next_batch(rows: list[Row], last_wm: int | None) -> tuple[list[Row], int | None]:
    """Rows above the watermark, and the new watermark.

    Args:
        rows: Source rows, each with an integer rv.
```

```
        last_wm: Largest rv copied so far, or None before the first run.

    Returns:
        The rows to copy and the watermark to store.
    """
    batch = [r for r in rows if last_wm is None or r["rv"] > last_wm]
    if not batch:
        return [], last_wm
    return batch, max(r["rv"] for r in batch)
```

Strictly greater avoids duplicates, and the empty-run guard keeps the watermark stable. The late-row gap remains, which is why CDC or ROWVERSION matters.

C4-L2 exercise 5

Hints. (1) Deletes matter here. (2) One row per key. (3) How do you recover missed changes?

Answer.

- CDC captures inserts, updates and deletes, with no watermark blind spots.
- Merge keeps one current row per order.
- Reset to a full copy for the table when you know changes were missed.

CDC plus Merge keeps silver in step with the source, and a per-table reset repairs known gaps.

Common wrong answers.

- method = `wm_modified`: A watermark misses deletes and late rows.
- update = `append`: Append adds each changed order again, breaking one row per key.
- recovery = `wait`: Changes below the stored state are not picked up by later runs.

C4-L3 Pipelines that fail fast, retry sensibly and alert

C4-L3 exercise 1

Hints. (1) It is much longer than most jobs. (2) Half a day. (3) 12 hours.

Answer. c) 12 hours Because: r1) The documented default timeout is 12 hours. Seven days is the maximum you can set.

12 hours by default. Set each activity's timeout close to its real run time, with some headroom.

Common wrong answers.

- `a`: 30 seconds is the default retry interval. The default timeout is 12 hours.
- `d`: Seven days is the maximum, not the default.
- `b`: The default is 12 hours.
- `c|r2`: Right answer, wrong reason. Timeouts are unrelated to capacity pausing.

C4-L3 exercise 2

Hints. (1) Four branches: success, failure, completion, skip. (2) Completion is not a failure. (3) Either outcome.

Answer. c) Whether the notebook succeeds or fails Because: r1) Upon Completion runs after success or failure. Upon Failure is the failure-only branch.

A failure alert on Upon Completion fires every night. Use Upon Failure.

Common wrong answers.

- `a`: That is Upon Failure.
- `b`: That is Upon Success.
- `d`: That is Upon Skip.
- `c|r2`: Right answer, wrong reason. It refers to this activity finishing, either way.

C4-L3 exercise 3

Hints. (1) Inside a string, expressions are wrapped in @{ }. (2) A whole value that must stay a number starts with @ alone. (3) `@{` and `@`.

Answer.

```
1 Folder path: landing/@{pipeline().parameters.wave}/
2 Max rows:    @pipeline().parameters.max_rows
```

`@{...}` inside text interpolates to a string. A bare `@...` expression keeps the value's type.

Common wrong answers.

- g2 = `@{`: String interpolation always returns a string. For a number, use the bare @ expression.

C4-L3 exercise 4

Hints. (1) A regular expression with fullmatch validates the shape. (2) Check each field's range. (3) Cap the total at 7 × 24 × 3600 seconds.

Answer.

```python
import re

TIMEOUT_PATTERN = re.compile(r"(\d+)\.(\d{2}):(\d{2}):(\d{2})")
MAX_SECONDS = 7 * 24 * 3600


def parse_timeout(value: str) -> int:
    """Seconds in a pipeline activity timeout written as D.HH:MM:SS.

    Raises:
        ValueError: On a bad format, out-of-range fields, or more than seven days.
    """
    match = TIMEOUT_PATTERN.fullmatch(value)
    if match is None:
        raise ValueError(f"timeout must be D.HH:MM:SS, got {value!r}")
    days, hours, minutes, seconds = (int(g) for g in match.groups())
    if hours > 23 or minutes > 59 or seconds > 59:
        raise ValueError(f"field out of range in {value!r}")
    total = days * 86400 + hours * 3600 + minutes * 60 + seconds
    if total > MAX_SECONDS:
        raise ValueError("timeout exceeds the seven-day maximum")
    return total
```

A linter that parses timeouts can flag every activity still on the 12-hour default, or set far above its normal run time.

C4-L3 exercise 5

Hints. (1) Which branch lets gold run after a failure? (2) Completion means either outcome. (3) Use Upon Success.

Answer. The bug is on line 3.

Fixed version:

```
Activity: nb_silver_orders
  Timeout: 0.00:30:00
Activity: nb_gold_sales  depends on nb_silver_orders: Upon Success
Activity: email_on_failure  depends on nb_silver_orders: Upon Failure
```

Upon Completion runs gold after a failed silver, publishing stale or partial data. Chain data steps on Upon Success.

Common wrong answers.

- Line 4: The alert is correctly on Upon Failure.
- Line 2: A 30-minute timeout is sensible for a 10-minute notebook.